Deploying Spring Boot application to the cloud

Not cover in this lab: Kubernetes clusters, complex cloud networking, paid cloud infrastructure.

What you will learn

- Package the entire Spring Boot application into a standalone executable JAR file.
- Push your project code cleanly to a GitHub remote repository.
- Provision a free, production-ready cloud MySQL database on Aiven.io (no credit card required).
- Adopt 12-Factor App principles: configure environment variables for cloud credentials without changing Java code.
- Create and deploy a public Docker Web Service on Render.com linked directly to your GitHub repository.
- Verify your live web application running on a public HTTPS URL accessible from any computer or mobile device.
- Clean up cloud services on Render and Aiven after evaluation to avoid unexpected costs or exhausted trial quotas.

The thrill of going live

Running on localhost is only the first half of software engineering. Deploying to the cloud transforms your project into a real, live internet product with a public HTTPS link that you can share with friends, teachers, and recruiters.

✓ Step 1. Package application into executable JAR

Open your terminal in the project root folder. Use the Maven Wrapper to build the executable fat JAR that bundles your compiled classes, Thymeleaf templates and embedded Tomcat server.

Terminal (Windows / macOS / Linux)

```
# On Windows (PowerShell or CMD):
.\mvnw.cmd clean package -DskipTests

# On Linux or macOS:
./mvnw clean package -DskipTests
```

- clean: removes old build artifacts from the target/ directory.
- package: compiles source code, packages resources, and produces target/todolist-0.0.1-SNAPSHOT.jar.
- -DskipTests: skips unit tests during build to speed up packaging.
- You can test running this JAR locally with: java -jar target/todolist-0.0.1-SNAPSHOT.jar

✓ Step 2. Create Multi-Stage Dockerfile and Push Project to GitHub

Initialize a Git repository (if not already done), verify your .gitignore file excludes target/ and secrets and push to GitHub.

.gitignore (verify target/ is ignored)

```
# Build output
target/
!.mvn/wrapper/maven-wrapper.jar

# IDE and OS files
.idea/
*.iml
.vscode/
.DS_Store
```

Render runs modern Java applications via Docker containers. Create a file named 'Dockerfile' (no file extension) in your project root directory next to the pom.xml file.

Dockerfile (in project root directory)

```
# Stage 1: Build the application using Maven (match <java.version> in pom.xml)
FROM maven:3-eclipse-temurin-17 AS build
# For Java 25: FROM maven:3-eclipse-temurin-25 AS build
# For Java 26: FROM maven:3-eclipse-temurin-26 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Minimal runtime image for production
FROM eclipse-temurin:17-jre-jammy
# For Java 25: FROM eclipse-temurin:25-jre
# For Java 26: FROM eclipse-temurin:26-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Matching Java Versions (Java 17, 21, 25, or 26)

The Temurin image tag in your Dockerfile must match the <java.version> declared in your pom.xml.
If your project was generated with Java 25 or 26, change '17' to '25' or '26' (e.g. maven:3-eclipse-temurin-25 and eclipse-temurin:25-jre) to prevent 'release version not supported' compilation errors.

Terminal (Git commands)

```
git init
git add .
git commit -m "feat: complete Todo application ready for deployment"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME.git
git push -u origin main
```

- Create a new public or private repository on https://github.com named todolist-spring-boot.
- Ensure maven-wrapper files (.mvn/ and mvnw) are committed; Render uses them to build your app in the cloud.
- Never commit passwords or secret API keys to GitHub.

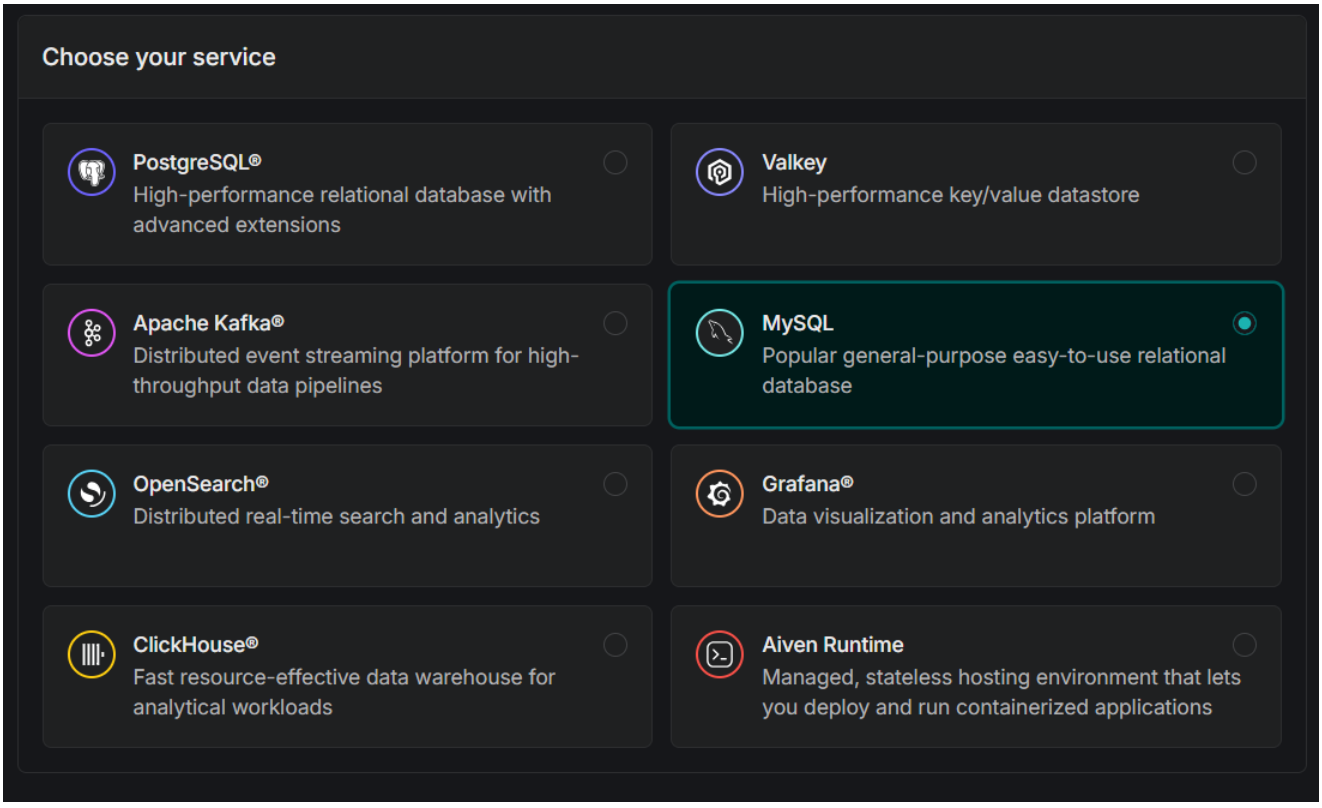
✓ Step 3. Provision free cloud MySQL on Aiven.io

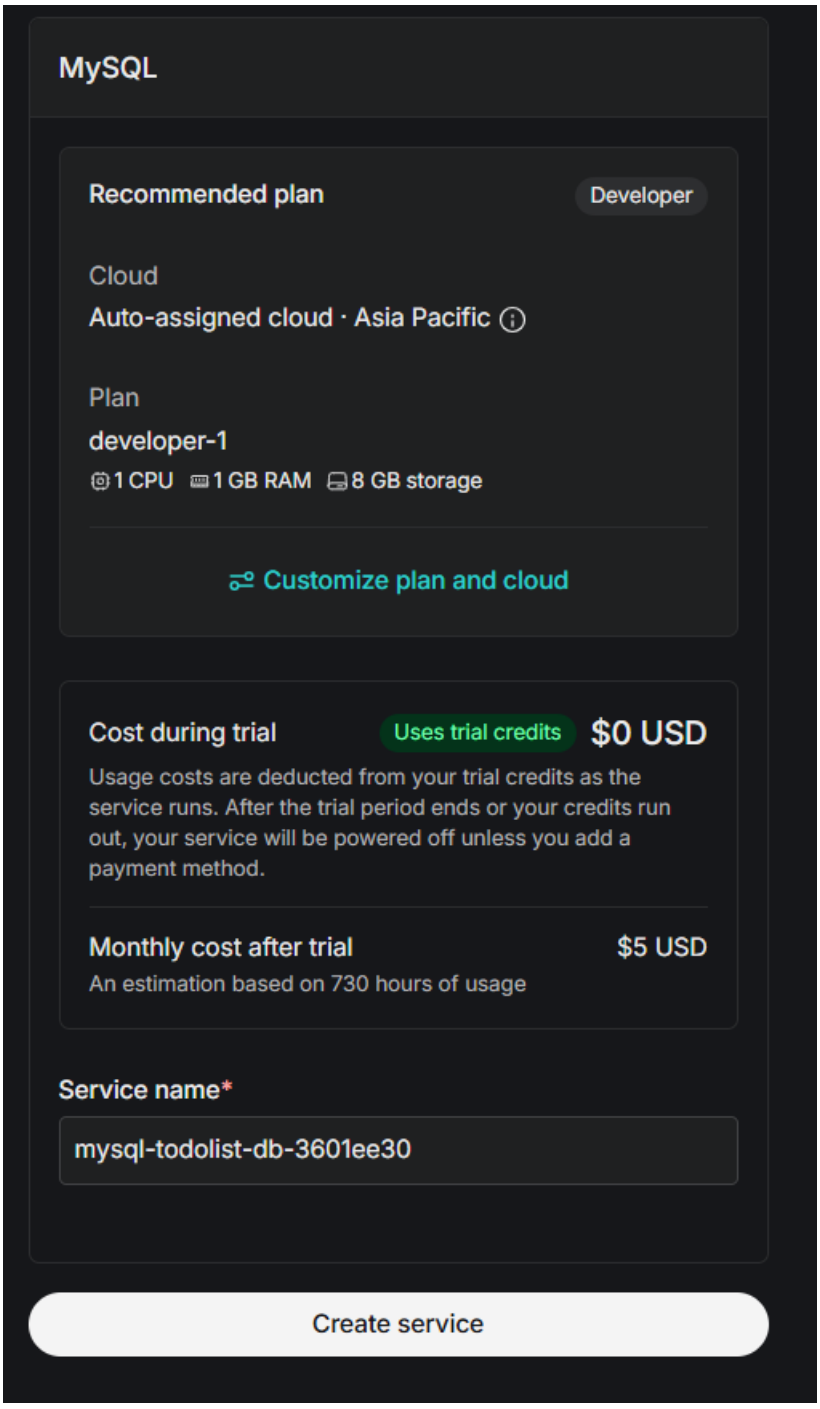
Create a free cloud-hosted MySQL database on https://aiven.io. Aiven provides a free Hobbyist tier with automated backups, SSL encryption and no credit card required.

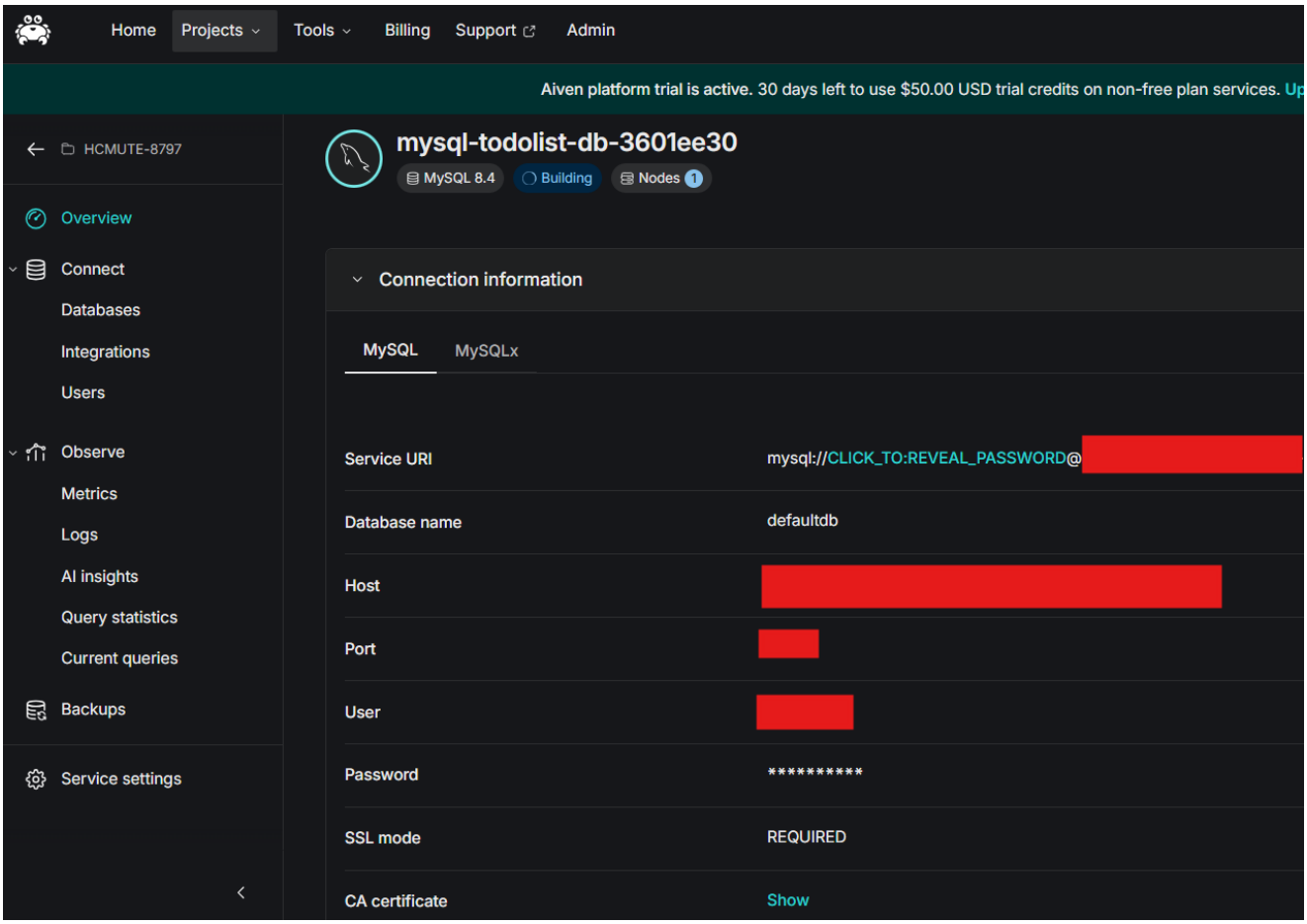
1. Go to https://aiven.io and sign up for a free account.
2. Click 'Create service' -> select 'MySQL' as the database engine.
3. Choose the 'Free Plan' (Hobbyist tier).
4. Select Cloud provider and Region: choose AWS or Google Cloud in Singapore (ap-southeast-1) for lowest latency from Vietnam.
5. Set Service name: 'mysql-todolist-db-3601ee30' and click 'Create service'.
6. Wait 1 to 2 minutes for status to change from 'Rebuilding' to 'Running'.

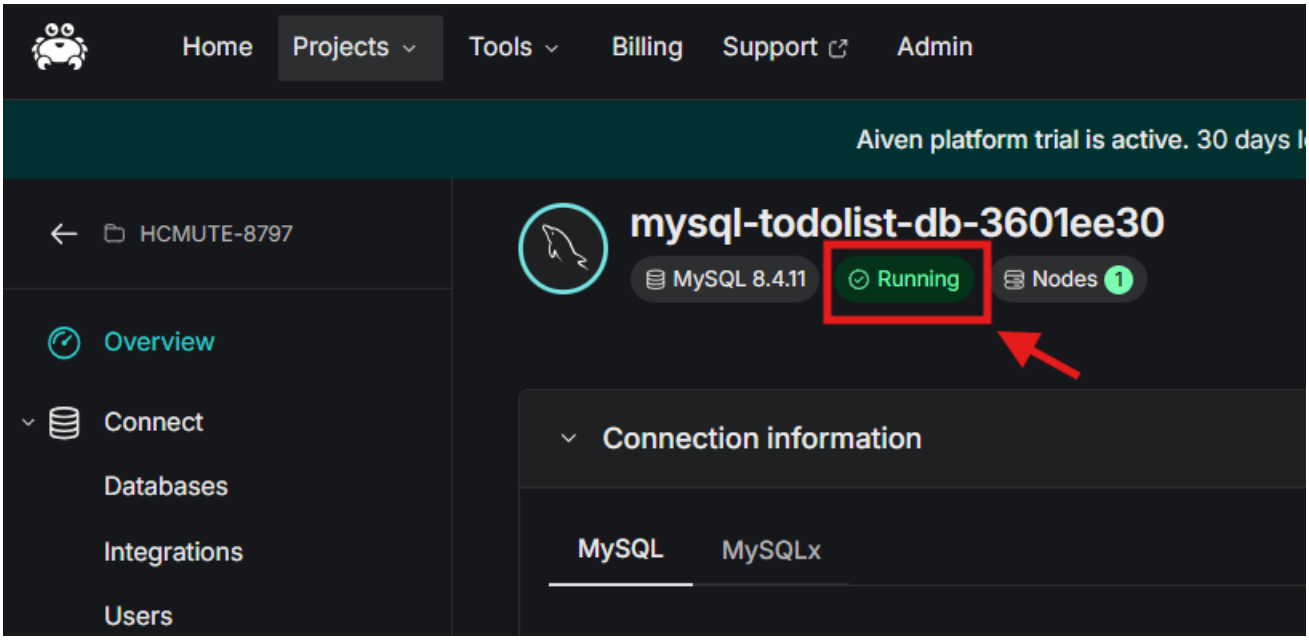
In the Aiven Service Overview tab, locate and note your Connection Information:

- Host: e.g., mysql-xxxx.aivencloud.com
- Port: e.g., 12345 (Aiven assigns a custom high port, NOT default 3306)
- User: avnadmin
- Password: click 'Copy' to get the generated strong password
- Database name: defaultdb
- Notice: Aiven enforces SSL encryption; append '?sslmode=require' to your JDBC connection string.




MySQL
Recommended plan
Developer
Cloud
Auto-assigned cloud · Asia Pacific
Plan
developer-1
1 CPU
1 GB RAM
8 GB storage
Customize plan and cloud
Cost during trial
Uses trial credits
$0 USD
Usage costs are deducted from your trial credits as the service runs. After the trial period ends or your credits run out, your service will be powered off unless you add a payment method.
Monthly cost after trial
$5 USD
An estimation based on 730 hours of usage
Service name*
mysql-todolist-db-3601ee30
Create service





Please wait until the status changes to 'Running'.

✓ Step 4. Configure environment variable overrides in application.properties

Update src/main/resources/application.properties. Use Spring placeholder syntax ${VAR:default} so your app runs locally with localhost defaults while allowing cloud environment variables to override them seamlessly.

src/main/resources/application.properties

```
# DataSource with Environment Variable overrides and local fallbacks
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:3306/todolist_db?createDa
tabaseIfNotExist=true&useSSL=false&serverTimezone=UTC}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:root}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:123456}

# Server port dynamically provided by Render
server.port=${PORT:8080}

# Hibernate DDL and SQL logging
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
```

- ${SPRING_DATASOURCE_URL:...}: Spring Boot checks for an environment variable named SPRING_DATASOURCE_URL first; if absent, it falls back to the local MySQL string.
- ${PORT:8080}: Render assigns a dynamic port via the PORT environment variable. This ensures your app listens on the exact port Render routes to.
- Commit and push this change to GitHub: git commit -am 'config: support cloud env variables' && git push

✓ Step 5. Deploy web service on Render.com

Render is a modern cloud hosting platform offering free hosting for web services connected directly to GitHub repositories.

1. Go to https://render.com and sign in with your GitHub account.
2. From the Dashboard, click 'New +' and select 'Web Service'.
3. Choose 'Build and deploy from a Git repository' and select your 'todolist-spring-boot' repository.
4. Configure Basic Settings: Name = 'todolist-app', Region = 'Singapore (Southeast Asia)', Branch = 'main', Language = 'Docker'.
5. Notice: Render automatically detects your Dockerfile so Build and Start commands are handled directly by container instructions.
6. Instance Type: select 'Free'.

Scroll down to 'Environment Variables' and click 'Add Environment Variable' for each of the following:

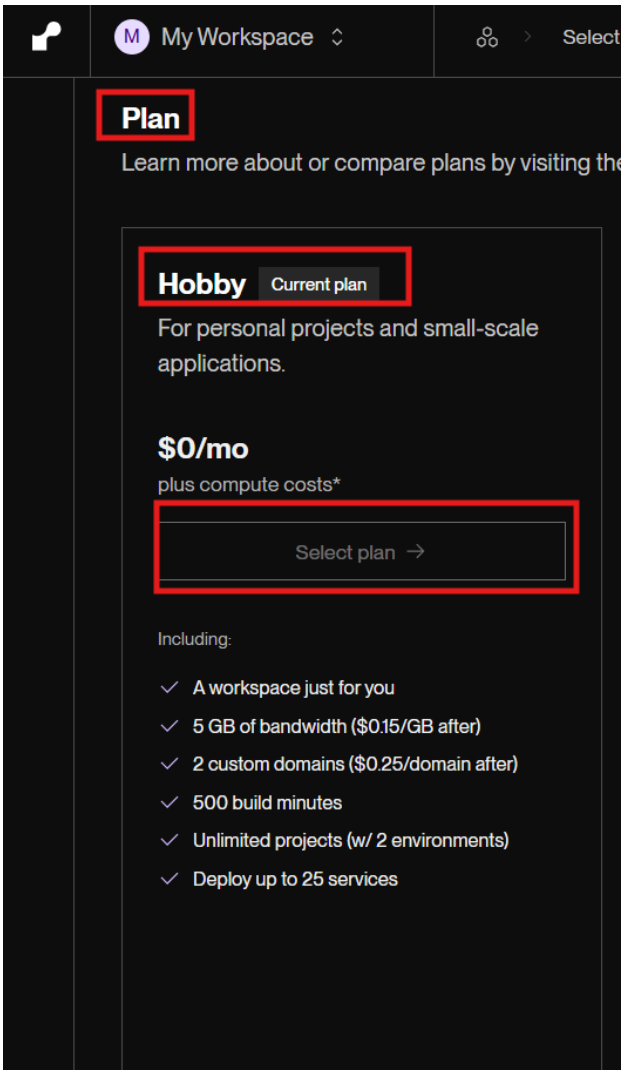
Render Environment Variables configuration

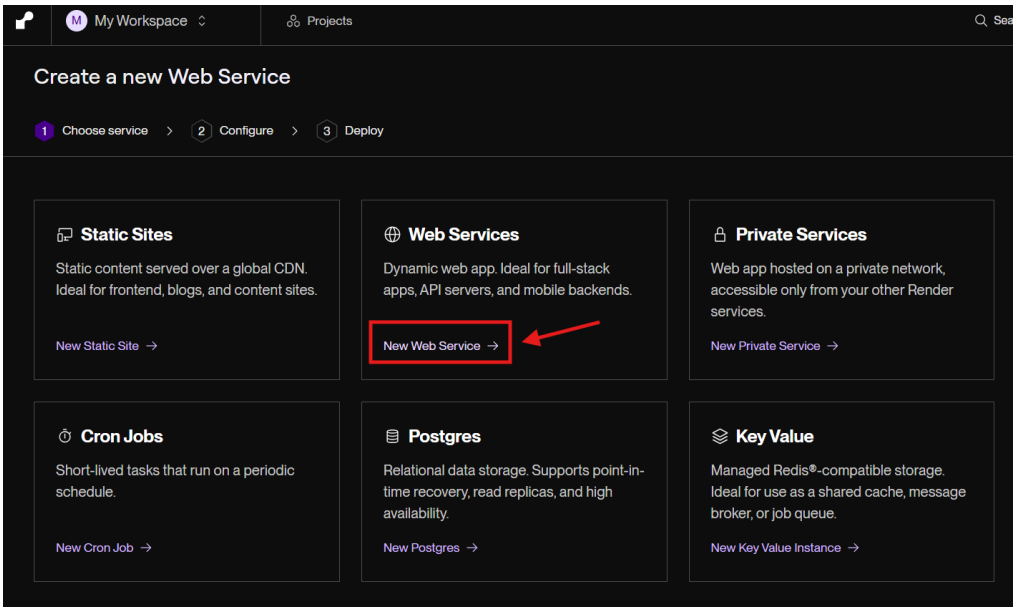
```
Key:   SPRING_DATASOURCE_URL
Value:
jdbc:mysql://YOUR_AIVEN_HOST:YOUR_AIVEN_PORT/defaultdb?createDatabaseIfNotExist=true&sslmode=r
equire

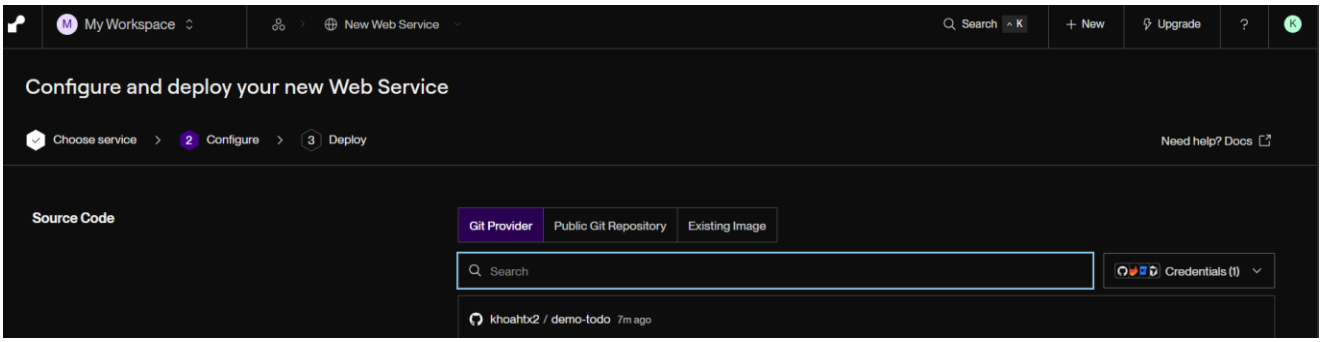
Key:   SPRING_DATASOURCE_USERNAME
Value: YOUR_AIVEN_USERNAME

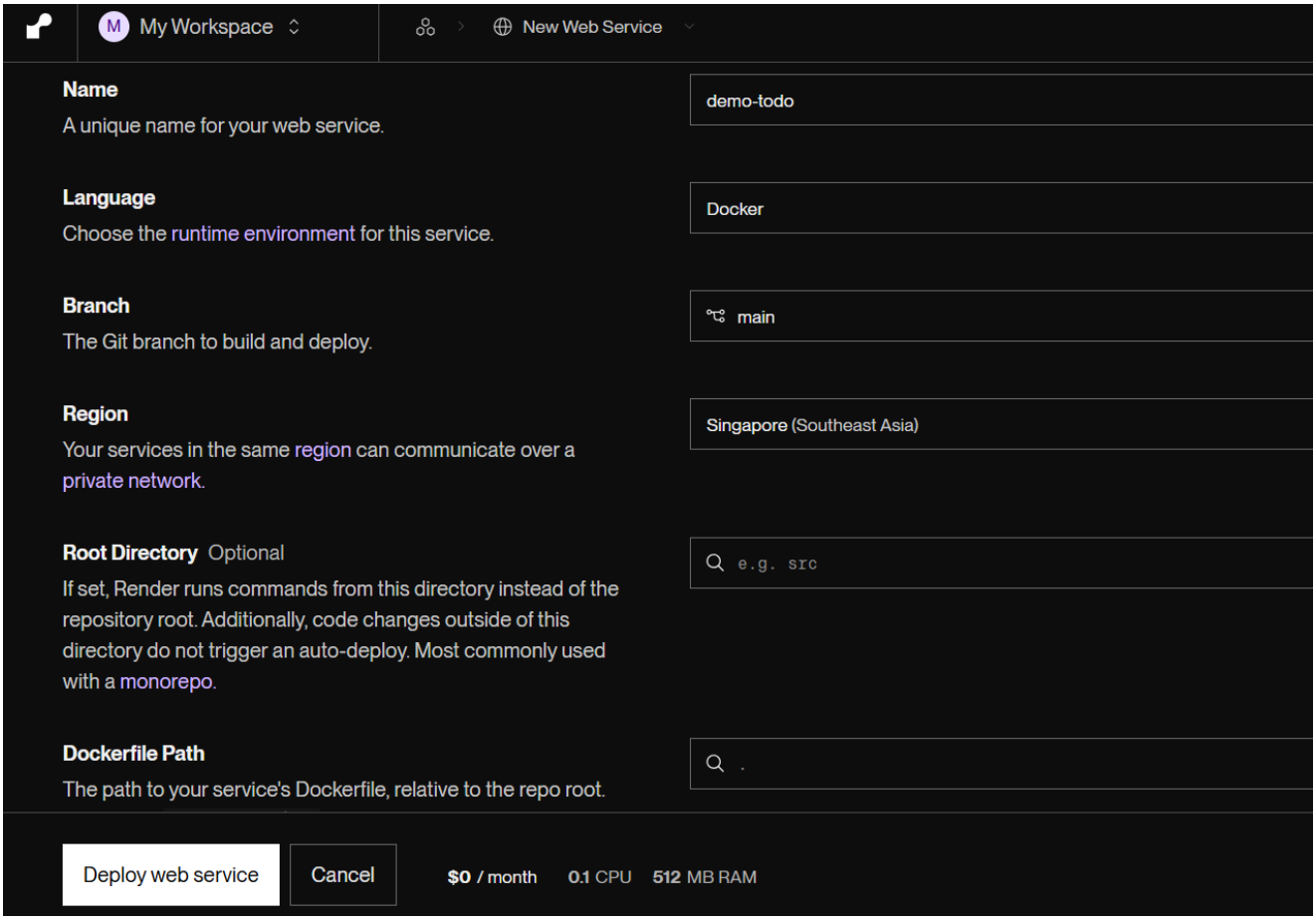
Key:   SPRING_DATASOURCE_PASSWORD
Value: YOUR_AIVEN_STRONG_PASSWORD
```

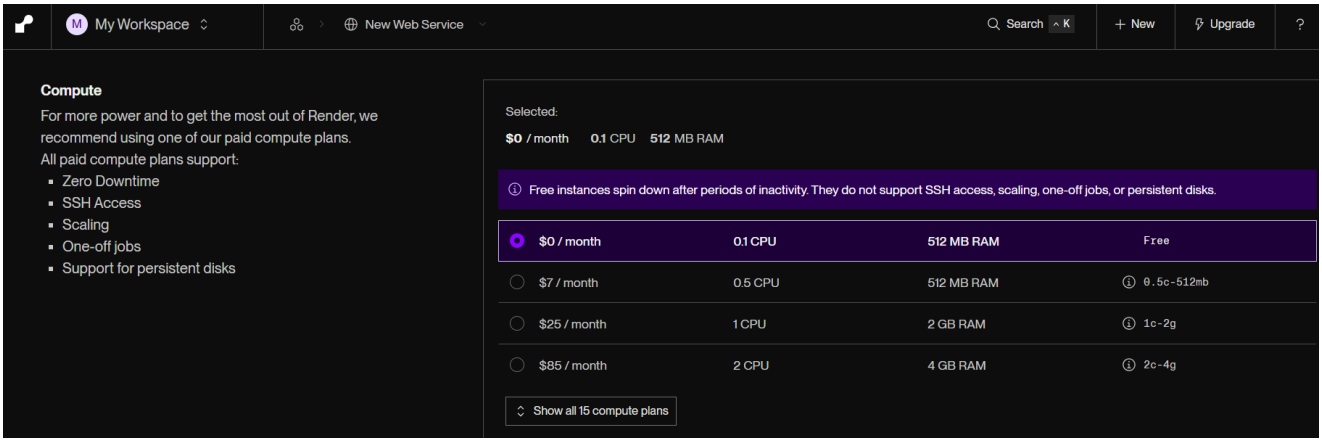
Click 'Deploy web service' button and Render will pull your code from GitHub repository, build the multi-stage Docker image and launch the containerized Spring Boot application.

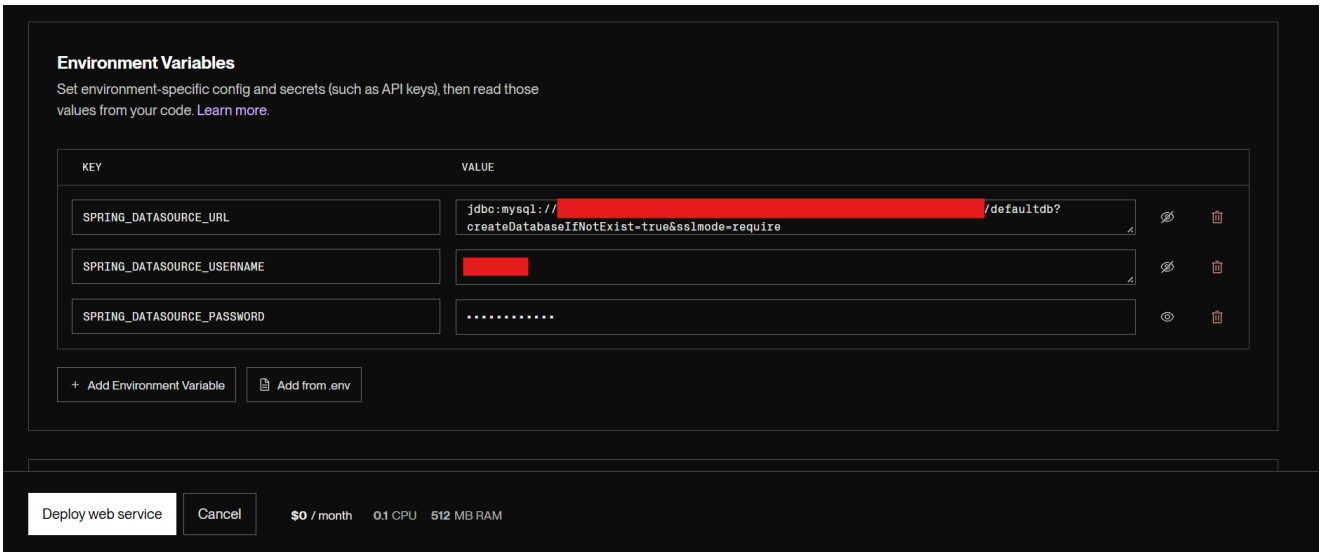

My Workspace
Select
Plan
Learn more about or compare plans by visiting the
Hobby
Current plan
For personal projects and small-scale applications.
$0/mo
plus compute costs*
Select plan →
Including:
A workspace just for you
5 GB of bandwidth ($0.15/GB after)
2 custom domains ($0.25/domain after)
500 build minutes
Unlimited projects (w/ 2 environments)
Deploy up to 25 services

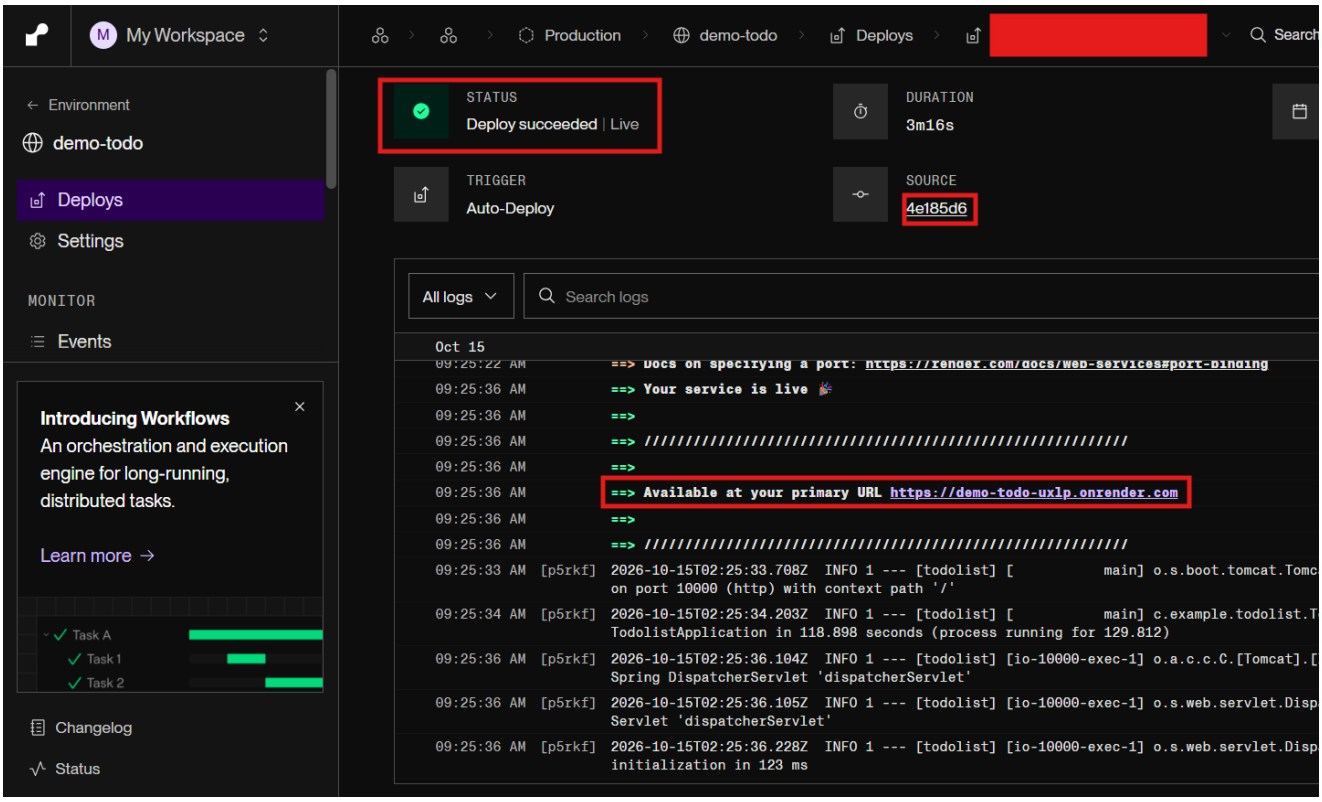

My Workspace
Projects
Create a new Web Service
1 Choose service
2 Configure
3 Deploy
Static Sites
Static content served over a global CDN. Ideal for frontend, blogs, and content sites.
New Static Site →
Web Services
Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.
New Web Service →
Private Services
Web app hosted on a private network, accessible only from your other Render services.
New Private Service →
Cron Jobs
Short-lived tasks that run on a periodic schedule.
New Cron Job →
Postgres
Relational data storage. Supports point-in-time recovery, read replicas, and high availability.
New Postgres →
Key Value
Managed Redis®-compatible storage. Ideal for use as a shared cache, message broker, or job queue.
New Key Value Instance →


My Workspace
New Web Service
Search K
New
Upgrade
Configure and deploy your new Web Service
Choose service
2 Configure
3 Deploy
Need help? Docs
Source Code
Git Provider
Public Git Repository
Existing Image
Search
Credentials (1)
khoahtx2 / demo-todo 7m ago


My Workspace
New Web Service
Name
A unique name for your web service.
demo-todo
Language
Choose the runtime environment for this service.
Docker
Branch
The Git branch to build and deploy.
main
Region
Your services in the same region can communicate over a private network.
Singapore (Southeast Asia)
Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo.
e.g. src
Dockerfile Path
The path to your service's Dockerfile, relative to the repo root.
.
Deploy web service
Cancel
$0 / month
0.1 CPU
512 MB RAM


My Workspace
New Web Service
Search K
New
Upgrade
Compute
For more power and to get the most out of Render, we recommend using one of our paid compute plans.
All paid compute plans support:
Zero Downtime
SSH Access
Scaling
One-off jobs
Support for persistent disks
Selected:
$0 / month 0.1 CPU 512 MB RAM
Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks.
$0 / month 0.1 CPU 512 MB RAM Free
$7 / month 0.5 CPU 512 MB RAM 0.5c-512mb
$25 / month 1 CPU 2 GB RAM 1c-2g
$85 / month 2 CPU 4 GB RAM 2c-4g
Show all 15 compute plans


Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. Learn more.
KEY
VALUE
SPRING_DATASOURCE_URL
jdbc:mysql://[redacted]/defaultdb?createDatabaseIfNotExist=true&sslmode=require
SPRING_DATASOURCE_USERNAME
[redacted]
SPRING_DATASOURCE_PASSWORD
••••••••••••
+ Add Environment Variable
Add from .env
Deploy web service
Cancel
$0 / month
0.1 CPU
512 MB RAM


My Workspace
Production
demo-todo
Deploys
Search
Environment
demo-todo
Deploys
Settings
MONITOR
Events
Introducing Workflows
An orchestration and execution engine for long-running, distributed tasks.
Learn more →
Task A
Task 1
Task 2
Changelog
Status
STATUS
Deploy succeeded | Live
DURATION
3m16s
TRIGGER
Auto-Deploy
SOURCE
4e185d6
All logs
Search logs
Oct 15
09:25:22 AM ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
09:25:36 AM ==> Your service is live 🎉
09:25:36 AM ==>
09:25:36 AM ==> ///
09:25:36 AM ==>
09:25:36 AM ==> Available at your primary URL https://demo-todo-uxlp.onrender.com
09:25:36 AM ==>
09:25:36 AM ==> ///
09:25:33 AM [p5rkf] 2026-10-15T02:25:33.708Z INFO 1 --- [todolist] [main] o.s.boot.tomcat.Tomc
on port 10000 (http) with context path '/'
09:25:34 AM [p5rkf] 2026-10-15T02:25:34.203Z INFO 1 --- [todolist] [main] c.example.todolist.T
TodolistApplication in 118.898 seconds (process running for 129.812)
09:25:36 AM [p5rkf] 2026-10-15T02:25:36.104Z INFO 1 --- [todolist] [io-10000-exec-1] o.a.c.c.C.[Tomcat].[
Spring DispatcherServlet 'dispatcherServlet'
09:25:36 AM [p5rkf] 2026-10-15T02:25:36.105Z INFO 1 --- [todolist] [io-10000-exec-1] o.s.web.servlet.Disp
Servlet 'dispatcherServlet'
09:25:36 AM [p5rkf] 2026-10-15T02:25:36.228Z INFO 1 --- [todolist] [io-10000-exec-1] o.s.web.servlet.Disp
initialization in 123 ms


demo-todo-uxlp.onrender.com/todos
Logged in as: admin ([ROLE_ADMIN])
Logout
Todo list
New todo
ID
Title
Due
Done
1
Todo 1
2026-10-31
No
Edit
Delete

✓ Step 6. Verify online deployment and cloud database

Monitor the live deployment logs in your Render dashboard and test your live website.

1. Watch the Render deployment console: look for the Spring Boot ASCII banner and 'Started TodolistApplication in X seconds'.
2. Copy your live public URL displayed under your service name: e.g., https://todolist-app-xxxx.onrender.com.
3. Open https://todolist-app-xxxx.onrender.com/todos in your browser: notice you are securely redirected to /login.
4. Log in with user credentials ('user' / 'password123') or admin credentials ('admin' / 'admin123').
5. Create a new task: 'Deploy Todo App to Render and Aiven' -> see it appear immediately in your browser!
6. Send the URL to a teammate or open on your smartphone: your web app is fully functional and live on the internet!

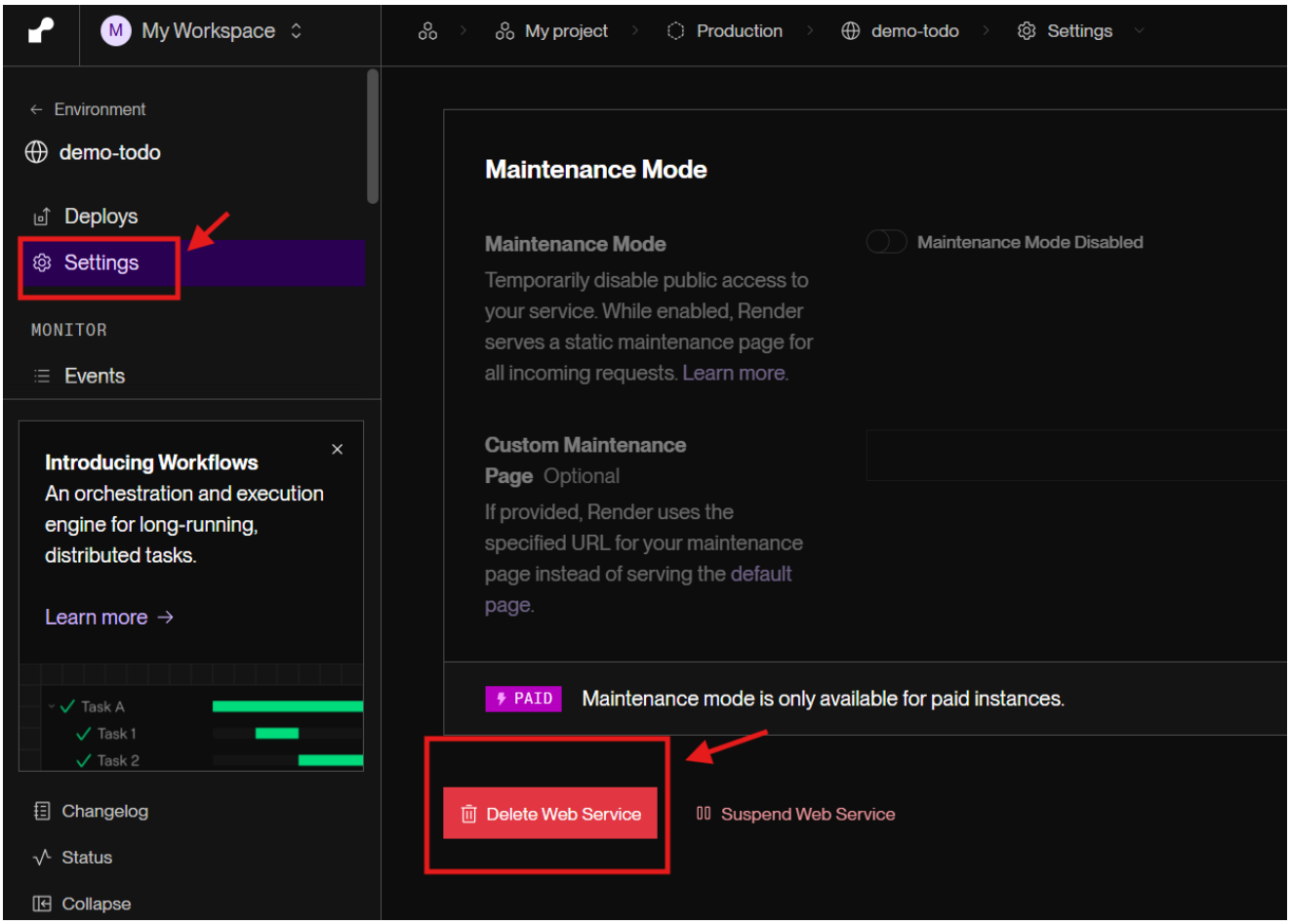
✓ Step 7. Clean up cloud resources to avoid unexpected charges

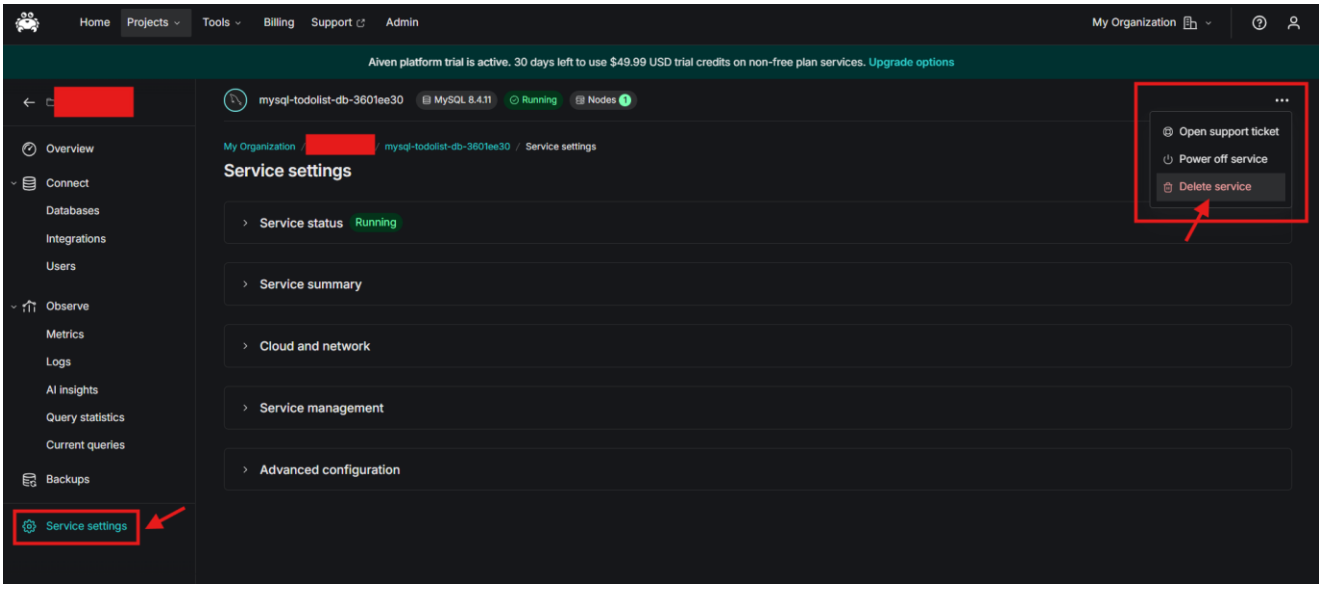
Cloud providers grant free hours and trial credits, but leaving idle services running permanently can quickly exhaust your free allocation or trigger billing once trial periods lapse. Once you have demonstrated or completed your lab, delete your cloud services.

1. Delete Web Service on Render: open your Render Dashboard -> click into 'todolist-app' -> select 'Settings' in the left menu -> scroll down to the 'Danger Zone' -> click 'Delete Web Service' -> type the service name and confirm deletion.
2. Delete MySQL Database on Aiven: go to https://console.aiven.io -> click on 'todolist-db' -> select 'Service settings' from the left sidebar -> scroll down to the 'Danger Zone' -> click 'Delete service' and confirm.
3. Verify zero active instances: refresh both dashboards to confirm no active services, running containers or billable instances remain.

Cloud Resource & Cost Hygiene

Shutting down or destroying test instances as soon as grading or testing is complete is a vital professional habit in cloud engineering. It eliminates unexpected credit-card charges and preserves free-tier quotas for your future coursework.





✓ Step 8. Summary

- Maven builds target/*.jar successfully using ./mvnw clean package.
- Multi-stage Dockerfile is added to project root to containerize the Spring Boot application.
- Free cloud MySQL database is active and running on Aiven.io.
- application.properties supports environment variables for datasource and port.
- Render web service is connected to GitHub and builds using the Docker runtime.
- Environment variables on Render point securely to the Aiven MySQL database.
- The public HTTPS URL renders the site and user can insert/update/delete/query todos in the cloud.
- Cloud resources on Render and Aiven are safely deleted after demonstration to prevent charges.